



UNIVERZITET U ZENICI
Politehnički fakultet u Zenici
Odsjek za softversko inženjerstvo
Napredne tehnike programiranja



Diplomski rad

PRIMJENA MAŠINSKOG UČENJA U DETEKCIJI PHISHING NAPADA

Adna Kos

Mentor:

Prof. dr. Nevzudin Buzađija

Zenica, 2026.

BIBLIOGRAFSKA KARTICA RADA

NAUČNO PODRUČJE RADA: Tehničke nauke

NAUČNO POLJE RADA: Elektrotehnika, elektronika i informaciono inženjerstvo

NAUČNA GRANA RADA: Softversko inženjerstvo

USTANOVA U KOJOJ JE RAD IZRAĐEN: Univerzitet u Zenici, Politehnički fakultet

MENTOR RADA: Prof. dr. Nevzudin Buzadija

DATUM ODBRANE RADA: 16.07.2026.

ČLANOVI KOMISIJE ZA ODBRANU:

Prof.dr. Samir Lemeš, predsjednik Komisije

Prof.dr. Nevzudin Buzadija, član Komisije i mentor

Prof.dr. Sabahudin Jašarević, član Komisije

IZJAVA

Izjavljujem da sam diplomski rad pod naslovom „Primjena mašinskog učenja u detekciji phishing napada" izradila samostalno pod mentorstvom Prof. dr. Nevzudina Buzadije. U radu sam koristila literaturu koja je navedena na kraju diplomskog rada. Tuđe spoznaje, zaključke, stavove, teorije i zakonitosti koje sam izravno ili parafrazirajući navela u diplomskom radu na uobičajan, standardan način sam povezala sa fusnotama i korištenim bibliografskim jedinicama.

Student:

Adna Kos

Sažetak

Phishing napadi predstavljaju jedan od najozbiljnijih oblika kibernetičkih prijetnji u savremenom digitalnom okruženju, uzrokujući značajne finansijske i reputacijske štete kako pojedincima tako i organizacijama širom svijeta. Prema izvještajima Anti-Phishing Working Group (APWG), globalne finansijske štete uzrokovane phishing napadima mjere se u desetinama milijardi dolara godišnje. Tradicionalni sistemi za detekciju phishinga, zasnovani na crnim listama i pravilima, pokazuju ograničenu efikasnost pri suočavanju s novim napadima jer su reaktivni po prirodi.

U ovom diplomskom radu istražena je primjena triju algoritama mašinskog učenja – Logistic Regression, Decision Tree i Random Forest – u automatskoj detekciji phishing URL-ova. Korišten je UCI Phishing Websites Dataset koji sadrži 11.055 primjera web stranica s 30 atributa. Skup podataka podijeljen je u omjeru 80/20 za treniranje i testiranje, a implementacija je realizirana u programskom jeziku Python koristeći biblioteke pandas, scikit-learn, scipy i matplotlib.

Rezultati eksperimentalne evaluacije pokazuju da Random Forest model postiže tačnost od 97,06%, preciznost od 97,31%, odziv od 96,02% i F1-score od 96,66%, nadmašujući Decision Tree (94,89%) i Logistic Regression (92,76%) u svim evaluiranim metrikama. Kros-validacijski F1-score Random Forest modela iznosi $0,9584 \pm 0,0031$. Analiza važnosti atributa identifikovala je SSL certifikat i ankore na eksternim domenama kao dominantne indikatore phishinga. Rad potvrđuje da mašinsko učenje, posebno ansambl metode poput Random Forest, predstavlja efikasnu alternativu klasičnim metodama detekcije phishinga.

Ključne riječi: phishing, mašinsko učenje, Random Forest, Logistic Regression, Decision Tree, kibernetička sigurnost, URL analiza.

ABSTRACT

Phishing attacks represent one of the most serious forms of cyber threats in the modern digital environment, causing significant financial and reputational damage to both individuals and organizations worldwide. According to reports from the Anti-Phishing Working Group (APWG), global financial losses caused by phishing attacks amount to tens of billions of dollars annually. Traditional phishing detection systems, based on blacklists and rule-based approaches, show limited effectiveness against new attacks due to their reactive nature.

This thesis investigates the application of three machine learning algorithms – Logistic Regression, Decision Tree, and Random Forest – in the automatic detection of phishing URLs. The UCI Phishing Websites Dataset was used, containing 11,055 samples of web pages with 30 attributes. The dataset was split in an 80/20 ratio for training and testing, and the implementation was carried out in Python using the pandas, scikit-learn, scipy, and matplotlib libraries.

The results of the experimental evaluation show that the Random Forest model achieves an accuracy of 97.06%, precision of 97.31%, recall of 96.02%, and F1-score of 96.66%, outperforming Decision Tree (94.89%) and Logistic Regression (92.76%) across all evaluated metrics. The Random Forest model's cross-validation F1-score is 0.9584 ± 0.0031 . Feature importance analysis identified the SSL certificate and anchors on external domains as the dominant phishing indicators. The thesis confirms that machine learning, particularly ensemble methods such as Random Forest, represents an effective alternative to classical phishing detection methods.

Keywords: phishing, machine learning, Random Forest, Logistic Regression, Decision Tree, cybersecurity, URL analysis.

Sadržaj

1. Uvod.....	1
2. Osnove informacione sigurnosti.....	2
2.1. Pojam informacione sigurnosti.....	2
2.2. Vrste sigurnosnih napada.....	3
2.3. Phishing napadi.....	4
3. Mašinsko učenje u sigurnosti.....	6
3.1. Osnove mašinskog učenja.....	6
3.2. Klasifikacijski algoritmi.....	6
3.3. Primjena ML u cyber sigurnosti.....	8
4. Pregled postojećih rješenja.....	10
5. Metodologija istraživanja.....	12
5.1. Dataset.....	12
5.2. Feature engineering.....	14
5.3. Modeli.....	15
5.4. Evaluacija.....	16
6. Eksperimentalni dio.....	17
6.1. Implementacija i struktura projekta.....	17
6.2. Okruženje i pokretanje projekta.....	19
6.3. Objašnjenje koda.....	20
6.4. Treniranje modela.....	24
6.4.1. Preprocesiranje podataka.....	25
6.4.2. Hiperparametarska optimizacija.....	26
6.5. Rezultati.....	28
7. Diskusija rezultata.....	34
8. Zaključak.....	37
9. Preporuke za praktičnu primjenu.....	39
9.1. Preporuke za implementaciju u produkcijskom okruženju.....	39
9.2. Preporuke za krajnje korisnike i organizacije.....	39
9.3. Etička i pravna razmatranja.....	40
10. Literatura.....	41

1. Uvod

Phishing napadi predstavljaju jedan od najozbiljnijih i najrasprostranjenijih oblika kibernetičkih prijetnji u savremenom digitalnom okruženju. Prema izvještajima Anti-Phishing Working Group (APWG), finansijske štete koje phishing napadi nanose globalnoj ekonomiji mjere se u desetinama milijardi dolara godišnje, pri čemu broj napada bilježi kontinuirani godišnji rast. Statistički podaci pokazuju da više od 90% uspješnih kibernetičkih upada u organizacije počinje upravo phishing napadom kao inicijalnim vektorom pristupa, što ovu prijetnju svrstava u strateški prioritet informacione sigurnosti na svim nivoima.

Phishing je oblik socijalnog inženjeringa u kojemu napadač lažno predstavlja legitimni entitet – banku, pružatelja e-pošte, državnu instituciju ili popularnu web stranicu s ciljem prevare žrtve da otkrije osjetljive podatke kao što su lozinka, broj kreditne kartice ili podaci za pristup korporativnim sistemima. Za razliku od sofisticiranih tehničkih eksploatacija, phishing ne zahtijeva visoko tehničko znanje već se oslanja na psihološke mehanizme manipulacije: urgentnost, strah i lažni autoritet.

Tradicionalni pristupi detekciji phishing napada oslanjaju se na crne liste URL-ova i domena (blacklists) te sustave zasnovane na poznatim potpisima napada (signature-based systems). Ovi pristupi imaju strukturalno ograničenje reaktivne prirode: phishing URL mora biti otkriven, prijavljen i verificiran prije nego što postane blokiran. Istraživanja pokazuju da između 80% i 90% novih phishing napada uspješno prolazi kroz crne liste u prvih nekoliko sati od pokretanja – upravo u periodu najveće aktivnosti napada.

Kao odgovor na ova ograničenja, istraživači su se okrenuli tehnikama mašinskog učenja (Machine Learning – ML). ML modeli uče obrasce iz historijskih podataka i sposobni su proaktivno prepoznati nove phishing stranice na osnovu karakteristika koje dijele s poznatim primjerima, bez potrebe za eksplicitnom identifikacijom napada.

Cilj ovog diplomskog rada jeste istražiti i usporediti efikasnost triju algoritama mašinskog učenja – Logistic Regression, Decision Tree i Random Forest – u kontekstu detekcije phishing URL-ova koristeći UCI Phishing Websites Dataset. Odabrana tri modela predstavljaju logičnu progresiju složenosti: od jednostavnog linearnog pristupa, preko nelinearnog modela zasnovanog na stablima, do sofisticirane ansambl metode. Implementacija je realizirana u Python programskom jeziku uz korištenje biblioteka scikit-learn, pandas i scipy.

2. Osnove informacione sigurnosti

2.1. Pojam informacione sigurnosti

Informaciona sigurnost (Information Security – InfoSec) je interdisciplinarno područje koje se bavi zaštitom informacija i informacijskih sistema od neovlaštenog pristupa, korištenja, otkrivanja, izmjene, narušavanja ili uništenja. U digitalnoj eri, gdje informacije predstavljaju jedan od najvrijednijih resursa organizacija i pojedinaca, informaciona sigurnost postala je strateški prioritet na svim nivoima – od individualnih korisnika do međunarodnih korporacija i državnih institucija.

Temelj informacione sigurnosti čini model poznat kao CIA trijada (Confidentiality, Integrity, Availability – povjerljivost, integritet, dostupnost). CIA trijada je uspostavljena kao temeljni okvir informacione sigurnosti još krajem 1970-ih i ostaje osnova većine savremenih sigurnosnih standarda i metodologija, uključujući ISO/IEC 27001 [1]. Ovaj model definira tri fundamentalne karakteristike koje svaki siguran informacijski sistem mora posjedovati.

Povjerljivost (Confidentiality) podrazumijeva da su informacije dostupne isključivo ovlaštenim osobama, procesima ili sistemima. Mehanizmi za osiguranje povjerljivosti uključuju kriptografiju (simetrično i asimetrično šifriranje), kontrolu pristupa zasnovanu na ulogama (RBAC), autentifikacijske protokole i virtualne privatne mreže (VPN). Phishing napadi direktno ugrožavaju povjerljivost jer im je cilj navesti žrtvu da dobrovoljno otkrije povjerljive informacije.

Integritet (Integrity) se odnosi na zaštitu informacija od neovlaštenih izmjena, osiguravajući da podaci ostanu tačni i konzistentni. Integritet je posebno kritičan u finansijskim transakcijama, medicinskim evidencijama i pravnim dokumentima. Mehanizmi zaštite uključuju kriptografske hash funkcije (SHA-256), digitalne potpise i kontrolne sume.

Dostupnost (Availability) garantira da su informacije i sistemi pristupačni ovlaštenim korisnicima u trenutku kada su im potrebni. Napadi koji ciljaju dostupnost uključuju distribuirane napade uskraćivanjem usluge (DDoS) te ransomware koji šifrira podatke čineći sisteme neupotrebljivim.

Uz CIA trijadu, savremena informaciona sigurnost uvodi i dodatne koncepte: neporecivost (non-repudiation) osigurava da niti jedna strana ne može poricati izvršenu akciju, što je posebno važno u elektronskom poslovanju; autentičnost (authenticity) garantira da su entiteti i podaci ono za što se predstavljaju, što je direktno relevantno za borbu protiv phishinga.

2.2. Vrste sigurnosnih napada

Kibernetički napadi klasificiraju se prema cilju napada (povjerljivost, integritet, dostupnost), tehničkom mehanizmu izvođenja ili motivaciji napadača. Prema prirodi djelovanja razlikujemo aktivne napade koji direktno narušavaju sistem i pasivne napade koji podrazumijevaju prisluškivanje bez izmjena.

Malware (zlonamjerni softver) je sveobuhvatni termin koji označava softver dizajniran za nanošenje štete. Uključuje viruse koji se šire zaražavanjem legitimnih fajlova, crve (worms) koji se autonomno repliciraju kroz mrežu, trojance koji se maskiraju kao legitimni softver, ransomware koji šifrira podatke i traži otkupninu, te spyware koji tajno prikuplja informacije o korisničkim aktivnostima. Distribucija malwarea često se oslanja na phishing e-mailove i lažne web stranice kao vektore isporuke. Sistematska klasifikacija zlonamjernog softvera prema mehanizmu širenja i cilju napada predstavlja osnovu za razumijevanje savremenih sigurnosnih prijetnji i dizajn odgovarajućih odbrambenih mehanizama. [2]

Napadi uskraćivanjem usluge (DoS/DDoS) imaju za cilj onesposobljavanje određenog servisa preopterećivanjem njegovih resursa. U distribuiranoj varijanti, napadač koordinira botnet mrežu kompromitiranih računara koji istovremeno šalju zahtjeve prema meti. Volumetrijski DDoS napadi mogu generirati saobraćaj od stotina gigabita u sekundi.

SQL injection napadi eksploatiraju ranjivosti u web aplikacijama koje ne provode adekvatnu validaciju korisničkog unosa. Napadač ubacuje zlonamjerni SQL kod u polja za unos koji se izvršava na bazi podataka, što može rezultirati neovlaštenim čitanjem, izmjenom ili brisanjem podataka.

Man-in-the-Middle (MitM) napadi podrazumijevaju da se napadač pozicionira između dvije legitimne strane u komunikaciji, presrećući i potencijalno mijenjajući poruke. Posebno su opasni u nezaštićenim Wi-Fi mrežama.

Ranjivosti nultog dana (Zero-day) su sigurnosni propusti u softveru koji su nepoznati proizvođaču u trenutku eksploatacije, zbog čega ne postoji zakrpa (patch) koja bi ih otklonila. Napadači koji otkriju takvu ranjivost mogu je iskoristiti prije nego što proizvođač uopšte sazna za njeno postojanje, što ove napade čini izuzetno opasnim i teško uočljivim tradicionalnim sigurnosnim alatima zasnovanim na potpisima poznatih prijetnji.

Credential stuffing i brute-force napadi oslanjaju se na automatizirano isprobavanje velikog broja kombinacija korisničkog imena i lozinke. Kod credential stuffing napada koriste se već procurjele kombinacije podataka za prijavu iz ranijih curenja podataka (data breaches), polazeći od pretpostavke da korisnici često ponovo koriste iste lozinke na više različitih servisa.

Watering hole napadi podrazumijevaju kompromitaciju legitimne web stranice koju često posjećuje ciljana grupa korisnika – na primjer zaposlenici određene organizacije ili industrije – u koju napadač ubacuje zlonamjerni kod. Za razliku od phishinga koji zahtijeva aktivnu interakciju žrtve s lažnom porukom, watering hole napad čeka da žrtva sama posjeti već kompromitiranu, inače povjerljivu stranicu.

Phishing napadi, opisani u narednoj sekciji, spadaju u kategoriju socijalnog inženjeringa i iskorištavaju psihološke slabosti korisnika umjesto tehničkih ranjivosti sistema.

2.3. Phishing napadi

Termin "phishing" nastao je 1990-ih godina kao kombinacija engleskih riječi "fishing" (ribolov) i "phreaking" (hakiranje telefonskih sistema). Metafora ribolova opisuje napadačevu strategiju: baca se primamljiva "udica", a naivni korisnici su "riba" koja se hvata. Phishing je oblik prevare u kojemu napadač kreira lažni identitet legitimne organizacije s ciljem navođenja žrtve da otkrije osjetljive podatke ili instalira malware.

Email phishing je najrasprostranjeniji oblik napada. Napadač šalje masovne e-mail poruke koje vizualno oponašaju komunikaciju legitimnih organizacija i pozivaju žrtvu na hitnu akciju – klik na link koji vodi do lažne stranice ili otvaranje priloženog dokumenta s malwareom. Poruke koriste psihološke taktike: urgentnost ("Vaš račun će biti zatvoren za 24 sata"), strah ("Detektirana je sumnjiva aktivnost") ili finansijsku privlačnost.

Spear phishing je ciljani oblik napada prema specifičnoj osobi ili organizaciji. Napadač unaprijed prikuplja informacije o žrtvi iz LinkedIn profila, web stranice kompanije i društvenih mreža, a potom konstruira personaliziranu poruku. Whaling je posebna varijanta usmjerena prema izvršnim direktorima i top menadžmentu (C-suite), a Business Email Compromise (BEC) napadi uzrokuju milijarde dolara štete godišnje.

URL phishing, centralna tema ovog istraživanja, podrazumijeva kreiranje lažnih URL adresa koje vizualno nalikuju legitimnim. Napadači koriste typosquatting (domene s namjernim tipfelima, npr. gooogole.com), homograph napade s vizualno sličnim Unicode znakovima, te dodavanje poddomena koji simuliraju legitimne organizacije (npr. paypal.secure-login.net). Karakteristike phishing URL-ova uključuju neobičnu dužinu, prisustvo znakova poput @, višestruke točke, IP adrese umjesto domenskih naziva te ključne riječi poput secure ili login u domenama trećih strana.

3. Mašinsko učenje u sigurnosti

3.1. Osnove mašinskog učenja

Mašinsko učenje (Machine Learning – ML) je podoblast vještačke inteligencije koja se bavi razvojem algoritama koji računarskim sistemima omogućavaju učenje iz podataka bez eksplicitnog programiranja za svaki korak. Termin je popularizirao Arthur Samuel 1959. godine, koji je ML opisao kao "sposobnost računara da uče bez eksplicitnog programiranja". Fundamentalna razlika u odnosu na klasično programiranje je u tome da sistem ne dobiva eksplicitna pravila – ona ih uči iz podataka.

Nadzirano učenje (Supervised Learning) temelji se na skupu označenih primjera – parovima (ulaz, izlaz) – na osnovu kojih algoritam uči funkciju mapiranja. Model prima ulazne podatke i poznate oznake, a iterativno podešava parametre kako bi minimizirao grešku predikcije. Detekcija phishinga spada u ovu kategoriju: svaki URL označen je kao phishing ili legitiman, a model uči granicu između ove dvije klase.

Nenadzirano učenje (Unsupervised Learning) radi s neoznačenim podacima, tražeći skrivene strukture bez prethodnog znanja o klasama. Algoritmi klasterisanja kao što su K-means i DBSCAN grupiraju slične primjere, dok metode redukcije dimenzionalnosti (PCA, t-SNE) smanjuju broj atributa uz očuvanje ključnih informacija. U kibernetičkoj sigurnosti koristi se za detekciju anomalija u mrežnom saobraćaju.

Polunadzirano učenje (Semi-supervised Learning) kombinira oba pristupa, koristeći mali broj označenih i veliki broj neoznačenih primjera. Posebno je korisno u sigurnosnim aplikacijama gdje je označavanje podataka skupo i zahtijeva ekspertsko znanje.

3.2. Klasifikacijski algoritmi

U kontekstu detekcije phishinga, zadatak se formulira kao binarna klasifikacija: svaki URL klasificira se u jednu od dvije klase – phishing ili legitimno. Za ovaj zadatak koriste se algoritmi nadgledanog učenja koji na osnovu trening podataka grade model za klasifikaciju novih, prethodno neviđenih primjera.

Logistička regresija (Logistic Regression) je statistički model koji se koristi za klasifikacijske zadatke unatoč svom nazivu. Razvio ga je statistički David Cox 1958. godine. Model procjenjuje vjerovatnost klase primjenom logističke (sigmoidne) funkcije: $P(y=1|x) = 1/(1 + e^{-(z)})$, gdje je $z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$ linearna kombinacija atributa. Regularizacija (L2 – Ridge, korištena u ovom radu) penalizira kvadrate koeficijenata sprječavajući prekomjerno učenje. Prednosti uključuju interpretabilnost i brzo treniranje; ograničenje je nemogućnost hvatanja nelinearnih odnosa. Logistička regresija koristi se kao baseline model.

Stablo odlučivanja (Decision Tree) je nelinearni algoritam koji gradi model u obliku hijerarhijskog stabla: svaki unutarnji čvor predstavlja test na jednom atributu (npr. "Je li dužina URL-a veća od 75 znakova?"), svaka grana odgovara mogućem ishodu, a listovi su konačne klasifikacijske odluke. CART algoritam koristi Gini indeks nečistoće za odabir optimalnog atributa pri grananju. Stabla su visoko interpretabilna – ekspert može pratiti put odlučivanja od korijena do lista. Sklona su prekomjernom učenju kada su duboka, što se kontrolira ograničenjem `max_depth=10`.

Slučajna šuma (Random Forest) je ansambl metoda koju su razvili Leo Breiman i Adele Cutler, objavivši rad 2001. godine koji je do danas citiran više od 80.000 puta. [3] Algoritam gradi 100 stabala odlučivanja i kombinira predikcije glasanjem. Uvodi dva izvora slučajnosti: bootstrap agregaciju (bagging) gdje se svako stablo trenira na nasumičnom podskupu podataka s vraćanjem, i nasumičan odabir $\sqrt{n}$ atributa pri svakom grananju. Ova kombinacija reducira varijancu modela bez povećanja sistematske greške (bias-variance tradeoff). Pored visoke tačnosti, Random Forest nudi procjenu važnosti atributa (feature importance) zasnovanu na prosječnom smanjenju Gini nečistoće.

Naivni Bayes (Naive Bayes) je probabilistički klasifikator koji se temelji na Bayesovom teoremu uz pretpostavku uvjetne nezavisnosti atributa: $P(klasa|podaci) \propto P(podaci|klasa) \times P(klasa)$. Unatoč simplificirajućoj pretpostavci koja je rijetko ispunjena u praksi, postiže iznenađujuće dobre rezultate posebno u klasifikaciji teksta i filtriranju neželjene pošte. Prednosti su izuzetno brzo treniranje, mali memorijski otisak i dobra skalabilnost.

Support Vector Machine (SVM) je algoritam koji traži optimalnu hiperravninu (decision boundary) koja maksimizira marginu između dvije klase. Uz primjenu kernel funkcija (npr. RBF kernel), SVM može modelirati i nelinearne granice odlučivanja projicirajući podatke u

prostor više dimenzionalnosti. U literaturi o detekciji phishinga SVM se često koristi kao referentni model za poređenje – naprimjer u istraživanju Basneta, Mukkamale i Sunga opisanom u poglavlju 4 – a njegova prednost je dobra generalizacija i na skupovima podataka s relativno malim brojem primjera.

k-najbližih susjeda (k-Nearest Neighbors, k-NN) je jednostavan algoritam koji klasificira novi primjer na osnovu klase njegovih k najbližih susjeda u prostoru atributa, najviše mjereno euklidskom udaljenošću. k -NN ne zahtijeva eksplicitnu fazu treniranja modela (tzv. lazy learning), već cijeli proces učenja odgađa do trenutka klasifikacije, što ga čini računski zahtjevnim za velike skupove podataka poput onog korištenog u ovom radu.

3.3. Primjena ML u cyber sigurnosti

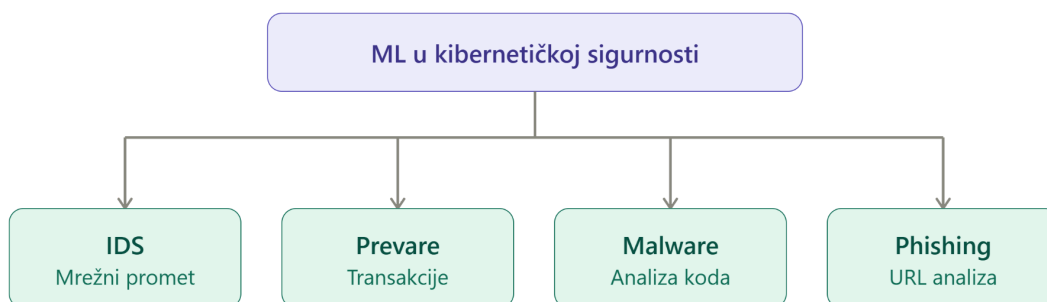
Mašinsko učenje postalo je nezaobilazan alat u modernoj kibernetičkoj sigurnosti zahvaljujući sposobnosti obrade velikih količina podataka i detektiranja suptilnih obrazaca koji izmaknu ručnoj analizi.

Sistemi za detekciju intruzija (IDS) koriste ML za razlikovanje normalnog od abnormalnog mrežnog prometa u realnom vremenu. Savremeni IDS sistemi poput Snort, Suricata i Zeek integriraju ML komponente koje klasificiraju mrežne pakete koristeći algoritme poput Random Forest, SVM i dubokih neuronskih mreža.

Detekcija prevara (Fraud Detection) u financijskim institucijama jedan je od najranijih primjera primjene ML u sigurnosti. Visa, Mastercard i PayPal koriste ML modele koji u realnom vremenu analiziraju stotine atributa svake transakcije i dodjeljuju ocjenu rizika. Ključni izazov je visoka neravnoteža klasa: legitimnih transakcija ima višestruko više od lažnih.

Analiza malwarea koristi ML za klasifikaciju nepoznatih programa kao bezopasnih ili zlonamjernih. Statička analiza ekstrahuje atribute iz binarnog koda bez izvršavanja, dok dinamička analiza prati sistemske pozive tokom kontroliranog izvršavanja u sandboxu.

Detekcija phishing URL-ova primjena je ML posebno relevantna za ovaj rad. Sistemi zasnovani na ML analiziraju kombinaciju atributa URL-a, sadržaja web stranice i mrežnih informacija. Google Safe Browsing, koji štiti milijarde korisnika Chrome preglednika, koristi ML modele za detekciju novih phishing stranica koje još nisu na crnoj listi.



Slika 1. Četiri ključna područja primjene mašinskog učenja u kibernetičkoj sigurnosti obrađena u ovom poglavlju.

Iako se svako od njih oslanja na različite tipove atributa i podatkovnih izvora – mrežni promet kod IDS sistema, transakcijske obrasce kod detekcije prevara, karakteristike izvršnog koda kod analize malwarea te URL i sadržajne attribute kod phishing detekcije – zajednička im je osnovna paradigma: model uči razlikovati normalno od anomalnog ponašanja na osnovu historijskih, označenih primjera, umjesto da se oslanja na ručno definirana pravila. Upravo ova zajednička paradigma predstavlja osnovu za pristup primijenjen u eksperimentalnom dijelu ovog rada (poglavlje 6), gdje se ista logika primjenjuje specifično na detekciju phishing URL-ova.

4. Pregled postojećih rješenja

Detekcija phishing napada evoluirala je kroz nekoliko generacija pristupa, od reaktivnih mehanizama zasnovanih na crnim listama do proaktivnih sistema zasnovanih na mašinskom učenju.

Tradicionalni pristupi: crne liste i potpisi

Crne liste (blacklists) su hronološki najstariji mehanizam detekcije phishing sadržaja. Organizacije kao što su Anti-Phishing Working Group (APWG), PhishTank, Google Safe Browsing i Microsoft SmartScreen održavaju baze podataka URL-ova identificiranih kao phishing resursi, integrirane u web preglednike i sigurnosne alate. Fundamentalno ograničenje je reaktivna priroda: phishing URL mora biti otkriven, verificiran i dodan u bazu – proces koji može potrajati od nekoliko sati do nekoliko dana. U tom periodu napadačev sadržaj ostaje aktivan bez zaštite. Signature-based sistemi pamte karakteristične nizove bita koji identificiraju poznate phishing sadržaje, ali su potpuno beskorisni za nove varijante.

Pionirski ML pristupi

Basnet, Mukkamala i Sung (2008) demonstrirali su primjenu ML za klasifikaciju phishing e-mailova koristeći SVM, Naivni Bayes i neuronske mreže. [4] Ovo je bio jedan od prvih sistematskih istraživačkih pokušaja primjene ML u detekciji phishinga. Autori su pokazali da ML modeli mogu postići tačnost iznad 90%, što je bilo značajno poboljšanje u odnosu na tadašnje rule-based sisteme.

Abdelhamid, Ayesh i Thabtah (2014) razvili su pristup zasnovan na asocijativnoj klasifikaciji za detekciju phishing web stranica. [5] Asocijativna klasifikacija kombinira algoritme rudarenja asocijacijskih pravila s klasifikacijom, generirajući interpretabilna pravila oblika "Ako atribut $A = X$ i atribut $B = Y \rightarrow \text{phishing}$ ". Ovaj pristup postiže tačnost od oko 94% i posebno je vrijedan jer generira ljudski razumljiva pravila.

Zhang, Hong i Cranor (2007) predstavili su CANTINA sistem koji koristi TF-IDF analizu sadržaja web stranica. CANTINA analizira tekstualni sadržaj i pretražuje web tražilice kako bi provjerio autentičnost domene, postižući stopu detekcije od oko 97%.

Sahingoz, Buber, Demir i Diri (2019) proveli su opsežno istraživanje primjene sedam ML algoritama za detekciju phishing URL-ova koristeći hibridne NLP attribute na skupu od milion URL-ova. Random Forest je postigao tačnost od 97,98% – jedna od najviših performansi prijavljenih u literaturi.

Rao i Pais (2019) predložili su framework koji kombinira URL-zasnovane, sadržajne i mrežne attribute. Njihov Random Forest model postigao je tačnost od 99,4% na skupu od 10.000 primjera, naglašavajući ključnu važnost mrežnih atributa poput starosti domene.

Verma i Das (2017) istraživali su koje informacije su dovoljne za pouzdanu klasifikaciju phishing URL-ova. [6] Rad pokazuje da sama URL adresa – bez analize sadržaja stranice – sadrži dovoljno informacija za visoku tačnost detekcije, što omogućava klasifikaciju u realnom vremenu.

Jain i Gupta (2019) predložili su pristup koji analizira isključivo hiperlinkove unutar HTML izvornog koda web stranice, razvrstane u dvanaest kategorija specifičnih atributa^[10]. Rješenje je u potpunosti implementirano na strani klijenta (client-side) i ne zahtijeva pristup uslugama trećih strana poput pretraživača, što ga čini pogodnim za primjenu u realnom vremenu. Autori izvještavaju o tačnosti detekcije iznad 99%, potvrđujući da atributi izvedeni iz strukture linkova unutar stranice – slični ankor atributima korištenim u ovom radu – spadaju među najinformativnije signale za razlikovanje phishing od legitimnih stranica.

Pregledom literature identificiraju se jasni trendovi: Random Forest konzistentno postiže visoke performanse, važnost odabira atributa jednako je kritična kao i odabir algoritma, a konceptijski drift ostaje otvoreni istraživački problem.

5. Metodologija istraživanja

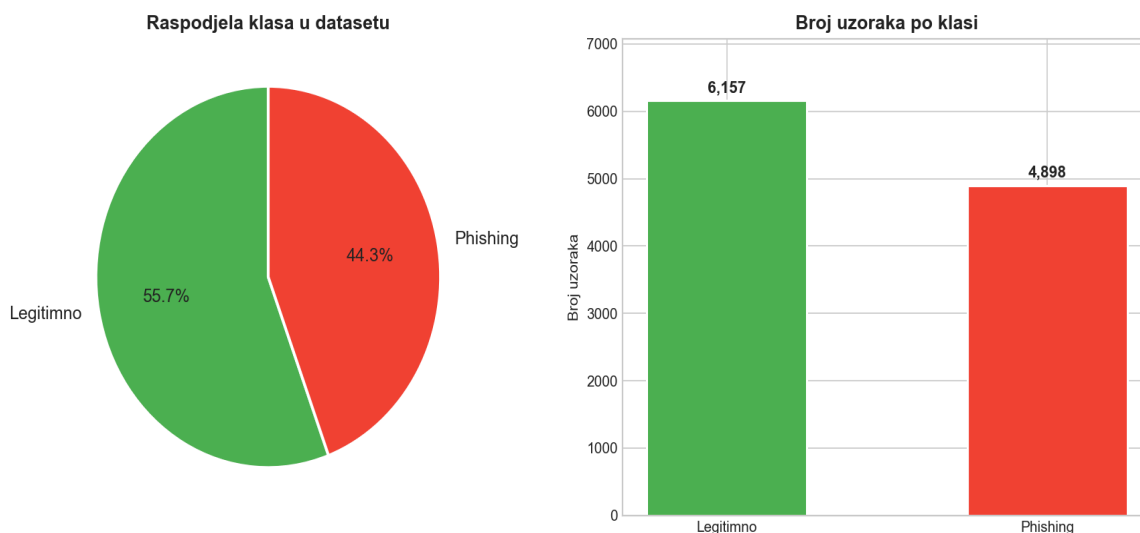
5.1. Dataset

Za eksperimentalni dio ovog rada korišten je UCI Phishing Websites Dataset, dostupan u UCI Machine Learning Repository. [7] Dataset je kreiran od strane Mohammad, Thabtah i McCluskey (2012) i od tada je korišten u stotinama istraživačkih radova, što ga čini de facto standardom za evaluaciju algoritama za detekciju phishinga.

Skup podataka sadrži ukupno 11.055 primjera web stranica, gdje svaki primjer predstavlja jednu web stranicu opisanu skupom od 30 ulaznih atributa i jednom ciljnom (izlaznom) varijablom *Result*. Provjerom kompletnosti podataka utvrđeno je da dataset ne sadrži nedostajuće (missing) vrijednosti ni u jednoj koloni, čime je izbjegnuta potreba za postupcima imputacije prije treniranja modela.

Svih 30 ulaznih atributa kodirano je trikategorički, sa vrijednostima isključivo iz skupa $\{-1, 0, 1\}$, gdje -1 označava jak indikator phishinga, 1 označava indikator legitimne stranice, a 0 predstavlja neutralnu/sumnjivu vrijednost. Atributi su grupisani u tri kategorije prema izvoru informacije: atributi zasnovani na URL adresi, atributi zasnovani na domeni i atributi zasnovani na sadržaju stranice (detaljno opisano u poglavlju 5.2). Dataset se distribuira u ARFF (Attribute-Relation File Format) formatu koji je razvio Weka alat za mašinsko učenje i funkcioniše slično kao CSV, ali sa opisom atributa na početku fajla. Za učitavanje je korištena `scipy.io.arff` biblioteka.

Potrebno je napomenuti da provjera identičnih redova pokazuje da se u datasetu javlja 5.206 kombinacija atributa koje se ponavljaju kod više različitih web stranica. Ovo nije greška u podacima, već očekivana posljedica činjenice da svaki od 30 atributa poprima samo jednu od tri diskretne vrijednosti (-1, 0, 1) – različite web stranice stoga mogu potpuno legitimno dijeliti identičan vektor atributa, jer sama URL adresa nije dio skupa atributa. Iz tog razloga, u skladu sa uobičajenom praksom pri radu s ovim datasetom u literaturi, ovi redovi nisu tretirani kao greška u prikupljanju podataka niti su uklonjeni iz analize.



Grafikon 1: Raspodjela klasa u UCI Phishing Websites Dataset (6.157 legitimnih, 4.898 phishing)

Ciljna varijabla *Result* je binarna i poprima vrijednosti -1 (phishing) i 1 (legitimno). Od ukupno 11.055 primjera, 6.157 primjera (55,69%) pripada klasi *legitimno*, dok 4.898 primjera (44,31%) pripada klasi *phishing*.

Omjer većinske i manjinske klase (imbalance ratio) iznosi $6.157 / 4.898 \approx 1,257 : 1$. Prema uobičajenoj klasifikaciji stepena neravnoteže korištenoj u literaturi o mašinskom učenju, dataset se smatra blago neuravnoteženim (mild imbalance) kada je omjer između 1:1 i približno 4:1, umjereno neuravnoteženim između 4:1 i 20:1, te ozbiljno neuravnoteženim (severe imbalance) preko 20:1. S obzirom da omjer u ovom datasetu ($\approx 1,26:1$) jasno spada u kategoriju blage neravnoteže, u radu nisu primijenjene tehnike balansiranja klasa poput SMOTE oversamplinga, undersamplinga ili *class_weight* parametra unutar modela – jer bi njihova primjena na ovako blago neuravnoteženom skupu podataka unijela nepotrebnu vještačku varijansu bez značajne koristi za performanse modela.

Prilikom podjele podataka u omjeru 80/20 (poglavlje 6.4), korišten je parametar *stratify=y*, čime je osigurano da omjer klasa iz originalnog dataseta bude vjerno očuvan u oba podskupa. Tabela 1a prikazuje stvarnu distribuciju klasa nakon stratificirane podjele, dobijenu direktnim izvršavanjem koda (*phishing_detection.py*) uz fiksiran *random_state=42*.

Skup podataka	Ukupno primjera	Legitimno	Phishing	Udio phishing klase
Kompletna dataset	11.055	6.157 (55,69%)	4.898 (44,31%)	44,31%
Trening skup	8.844	4.926 (55,70%)	3.918 (44,30%)	44,30%
Test skup	2.211	1.231 (55,68%)	980 (44,32%)	44,32%

Tabela 1. Distribucija klasa po skupovima podataka

Iz Tabele 1 vidljivo je da se udio phishing klase u trening skupu (44,30%) i test skupu (44,32%) praktično poklapa s udjelom u originalnom datasetu (44,31%), s odstupanjem manjim od 0,02 procenta poena. Ovo potvrđuje da stratificirana podjela nije unijela dodatnu neravnotežu i da je evaluacija modela na test skupu reprezentativna za distribuciju klasa u cijelom datasetu.

Zbog prisutne, iako blage, neravnoteže klasa, tačnost (Accuracy) sama po sebi ne predstavlja dovoljno pouzdanu mjeru uspješnosti modela. Naime, model koji bi za svaki primjer predvidio isključivo većinsku klasu (legitimno) postigao bi tačnost od 55,69% bez ikakve stvarne sposobnosti razlikovanja phishing i legitimnih stranica. Iz tog razloga su u radu, pored tačnosti, korištene i metrike preciznosti, odziva, F1-scorea i AUC vrijednosti (detaljno definisane u poglavlju 5.4), koje zajedno daju znatno potpuniju i pouzdaniju sliku performansi modela na obje klase, a posebno na manjinskoj (phishing) klasi čija je ispravna detekcija praktično najvažnija.

5.2. Feature engineering

Feature engineering je proces transformacije sirovih podataka u skup atributa najprikladnijih za zadatak učenja. 30 atributa UCI dataseta svrstavaju se u tri kategorije.

URL-zasnovani atributi obuhvataju karakteristike direktno izvedene iz URL adrese bez pristupa web stranici: dužinu URL-a (phishing URL-ovi su u prosjeku znatno duži od legitimnih), prisustvo simbola @ (browser ignorira sve ispred ovog znaka, što napadači eksploatiraju za maskiranje stvarne destinacije), korištenje IP adrese umjesto domenskog naziva (neuobičajeno za legitimne organizacije), dubinu URL puta te prisutnost crtica i višestrukih točaka u domenskom imenu.

Atributi zasnovani na domeni uključuju: starost domene (phishing domene tipično su registrirane kratko prije napada – manje od godinu dana, dok legitimne organizacije imaju dugo registrirane domene), dužinu registracije domene (legitimne organizacije plaćaju domene unaprijed na više godina), DNS zapis te prisutnost domene u Alexa rang listi najpopularnijih web stranica.

Atributi sadržaja stranice obuhvataju: SSL stanje certifikata (prisutnost i valjanost HTTPS veze), procenat ankora koji upućuju na vanjske domene (phishing stranice kopiraju HTML kôd legitimnih stranica zadržavajući originalne linkove), prisustvo iframe elemenata koji skrivaju sadržaj, te JavaScript funkcije za onemogućavanje desnog klika i statusne trake.

5.3. Modeli

U eksperimentalnom dijelu implementirana su tri klasifikacijska modela koja predstavljaju logičnu progresiju složenosti: Logistic Regression kao linearni baseline, Decision Tree koji uvodi nelinearne granice, te Random Forest kao ansambl metoda.

Logistic Regression konfigurisan je s L2 regularizatorom ($C=1.0$), solverom lbfgs i maksimalnim brojem iteracija od 1.000. Primijenjena je StandardScaler standardizacija koja centrira attribute oko nule i skalira na jediničnu standardnu devijaciju, jer je logistička regresija osjetljiva na razlike u skalama atributa.

Decision Tree implementiran je s Gini indeksom nečistoće (CART algoritam) i maksimalnom dubinom od 10 razina ($\text{max_depth}=10$) radi sprječavanja prekomjernog učenja. Random Forest konfigurisan je s 100 stabala ($\text{n_estimators}=100$), Gini kriterijem i automatskim odabirom $\sqrt{n}$ atributa pri grananju. Fiksiran seed slučajnosti $\text{random_state}=42$ primijenjen je za sve modele radi potpune reproducibilnosti eksperimenta.

5.4. Evaluacija

Performanse modela evaluirane su četirima standardnim metrikama binarne klasifikacije:

Tačnost (Accuracy) = $(TP + TN) / (TP + TN + FP + FN)$ – udio ispravno klasificiranih primjera. Preciznost (Precision) = $TP / (TP + FP)$ – udio stvarnih phishing stranica među svim primjerima koje je model označio kao phishing; visoka preciznost znači malo lažno pozitivnih rezultata. Odziv (Recall) = $TP / (TP + FN)$ – udio phishing stranica koje je model uspješno identificirao; visok odziv ključan je jer propuštene phishing stranice direktno ugrožavaju korisnike. F1-score = $2 \times (Precision \times Recall) / (Precision + Recall)$ – harmonijska sredina preciznosti i odziva.

Dodatno, ROC kriva (Receiver Operating Characteristic) i AUC vrijednost (Area Under Curve) koriste se kao dopunska mjera invarijantna na prag odlučivanja. Pored jednokratne train/test evaluacije, provedena je i 5-fold kros-validacija koja pruža robustniju procjenu generalizacijskih sposobnosti modela.

6. Eksperimentalni dio

Eksperimentalni dio ovog diplomskog rada realiziran je kao kompletan pipeline mašinskog učenja — od učitavanja sirovih podataka do vizualizacije i interpretacije rezultata. Cijela implementacija rađena je u programskom jeziku Python, koji je danas de facto standard u oblasti mašinskog učenja i analize podataka, zahvaljujući bogatom ekosistemu biblioteka, aktivnoj zajednici i ogromnoj količini dostupne dokumentacije. Python omogućava brzu iteraciju i eksperimentiranje, što ga čini idealnim za istraživačke projekte poput ovog.

Praktični cilj eksperimenta bio je odgovoriti na istraživačko pitanje: koji od triju odabranih algoritama mašinskog učenja — Logistic Regression, Decision Tree i Random Forest — postiže najbolje performanse u zadatku binarne klasifikacije phishing URL-ova, i kolika je razlika između njihovih performansi? Odabir upravo ovih triju modela motiviran je logičnom progresijom složenosti: Logistic Regression kao najjednostavniji linearni baseline, Decision Tree koji uvodi nelinearnost, i Random Forest kao sofisticirani ansambl koji kombinira prednosti stabala uz redukciju varijance.

6.1. Implementacija i struktura projekta

Implementacija je realizirana u programskom jeziku Python (verzija 3.10). Korištene biblioteke uključuju: `scipy` (v1.10) za učitavanje ARFF formata; `pandas` (v2.0) za manipulaciju tabelarnim podacima; `scikit-learn` (v1.3) za implementacije ML algoritama, metrika evaluacije i predobrade.[8] `numpy` (v1.24) za numeričke operacije; te `matplotlib` (v3.7) za vizualizaciju rezultata. Razvoj i testiranje koda provedeni su u standardnom desktop okruženju (operativni sistem neovisan o specifičnom hardveru, kod ne koristi GPU akceleraciju niti zavisi od operativnog sistema), što je uobičajeno za probleme klasifikacije nad tabelarnim podacima ovog obima (11.055 primjera, 30 atributa), gdje trening ni jednog od tri modela ne prelazi nekoliko sekundi na standardnom CPU-u. Cjelokupan kod organiziran je kao jedna samostalna, reproducibilna skripta (`phishing_detection.py`) koja pokriva čitav tok rada, od učitavanja sirovih podataka do generisanja finalnih grafikona bez potrebe za eksternim servisima ili mrežnim pristupom tokom izvršavanja.

Struktura projekta organizirana je na sljedeći način:

```

phishing_ml_project/
├── phishing_detection.py      ← glavni Python kod
├── requirements.txt          ← lista biblioteka
├── data/
│   └── phishing_dataset.arff ← UCI Phishing Websites Dataset
├── results/
│   ├── metrics_comparison.png ← grafički prikaz usporedbe modela
│   └── model_performance.csv  ← tabela rezultata u CSV formatu

```

Za pokretanje projekta potrebno je instalirati zavisnosti komandom `pip install -r requirements.txt`, a zatim pokrenuti `python phishing_detection.py`. Nakon pokretanja, u terminalu se pojavljuje tabela s rezultatima, otvara se prozor s grafom usporedbe metrika, a graf se automatski snima u `results/metrics_comparison.png`.

Sve zavisnosti navedene su u fajlu `requirements.txt` s minimalnim kompatibilnim verzijama, čime je osigurano da se okruženje može identično rekreirati na drugom računar — preduslov za reproducibilnost eksperimentalnih rezultata. Tabela 2 sistematizira sve korištene biblioteke, njihove verzije i konkretnu ulogu u implementaciji.

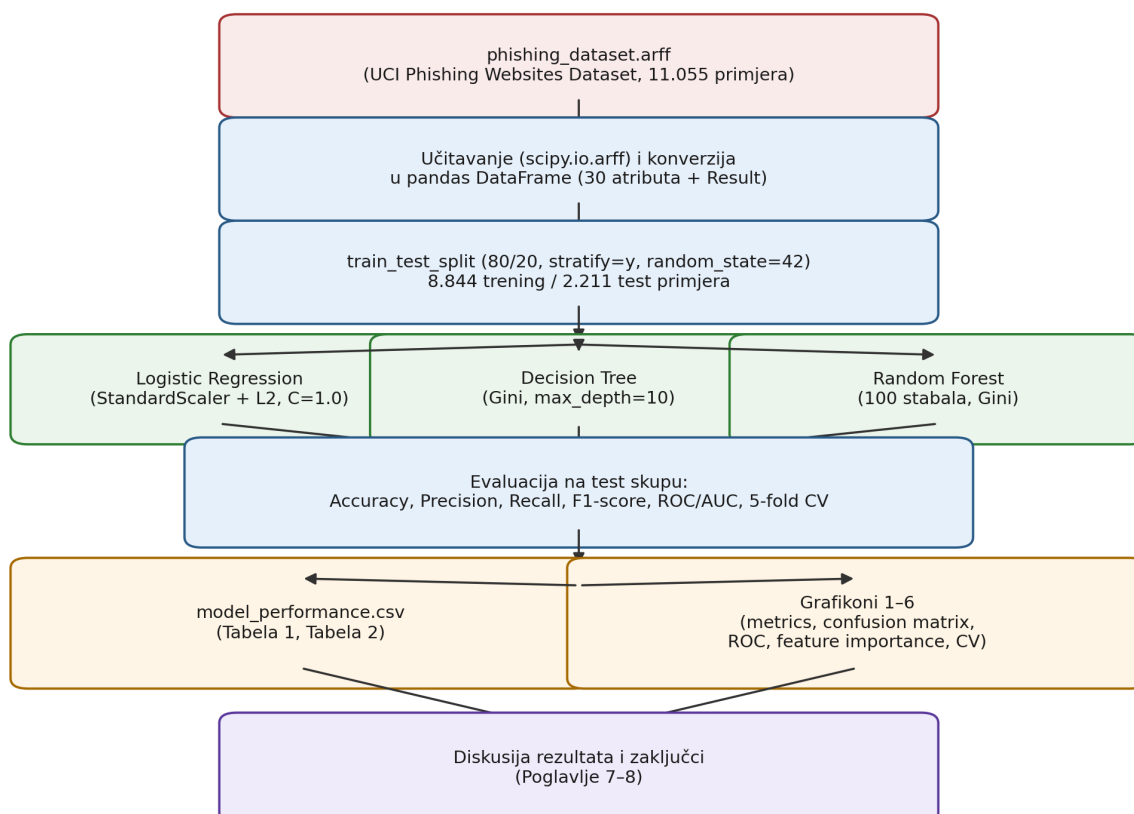
Biblioteka / alat	Verzija	Namjena u projektu
Python	3.10	Programski jezik u kojem je implementiran cijeli pipeline
scipy (scipy.io.arff)	1.10	Parsiranje i učitavanje .arff formata UCI dataseta
pandas	2.0	DataFrame struktura, transformacija tipova, selekcija kolona
scikit-learn	1.3	Implementacija LR/DT/RF modela, <code>train_test_split</code> , metrike, <code>StandardScaler</code>
numpy	1.24	Numeričke operacije nad nizovima i matricama (npr. <code>np.arange</code>)
matplotlib	3.7	Generisanje grafikona (Grafikon 1–6) i čuvanje u .png formatu

Tabela 2: Pregled korištenih tehnologija i biblioteka

6.2. Okruženje i pokretanje projekta

Pokretanje projekta od nule zahtijeva sljedeće korake, koji su korišteni i pri generisanju svih rezultata prikazanih u ovom radu:

- 1) Instalacija Python okruženja (verzija 3.10 ili novija) i kreiranje izoliranog virtualnog okruženja komandom `python -m venv venv`, čime se izbjegava sukob verzija biblioteka s drugim projektima na istom računaru.
- 2) Aktivacija virtualnog okruženja (`venv\Scripts\activate` na Windows sistemu, odnosno `source venv/bin/activate` na Linux/macOS sistemu).
- 3) Instalacija svih zavisnosti jednom komandom: `pip install -r requirements.txt`, koja instalira tačno one verzije biblioteka navedene u Tabeli 3.
- 4) Postavljanje UCI Phishing Websites Dataseta (`phishing_dataset.arff`) u poddirektorij `data/`, u skladu sa strukturom projekta prikazanom iznad.
- 5) Pokretanje glavne skripte komandom: `python phishing_detection.py`.



Slika 2. Arhitektura implementiranog ML pipeline-a — od sirovog dataseta do diskusije rezultata

Izvršavanje skripte traje svega nekoliko sekundi na standardnom prijenosnom računaru (bez potrebe za grafičkom karticom), s obzirom na relativno mali obim dataseta. Nakon pokretanja, u terminalu se ispisuje formatirana tabela s vrijednostima tačnosti, preciznosti, odziva i F1-scorea za sva tri modela (identična podacima prikazanim u Tabeli 1), potom se otvara interaktivni prozor s grafikonom usporedbe metrika, a isti grafikon se automatski snima na disk u results/metrics_comparison.png u rezoluciji od 150 DPI, pogodnoj za uključivanje u dokumentaciju ili izvještaje. Fiksiranje parametra random_state=42 pri svakom pozivu koji uključuje slučajnost (podjela podataka, inicijalizacija Random Forest stabala) osigurava da ponovljeno pokretanje skripte, na istom skupu podataka, uvijek proizvodi identične rezultate — što je ključan preduslov naučne reproducibilnosti eksperimenta.

6.3. Objašnjenje koda

U nastavku je prikazan kompletan Python kod s detaljnim objašnjenjima svakog koraka implementiranog ML pipeline-a.

Korak 1: Uvoz biblioteka

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.io import arff
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
```

NumPy se koristi za rad s numeričkim matricama i nizovima. Pandas pruža DataFrame strukturu za manipulaciju tabelarnim podacima, slično Excelu u kodu. Matplotlib/pyplot koristi se za crtanje grafova usporedbe metrika. Iz scipy.io uvozi se modul arff koji zna čitati .arff fajlove – format u kojemu je distribuiran UCI dataset. Iz scikit-learn uvozi se train_test_split za podjelu podataka, implementacije sva tri modela te četiri funkcije za evaluaciju performansi.

Korak 2: Učitavanje i obrada dataseta

```
# 1. Učitavanje dataseta
```

```
data, _ = arff.loadarff("data/phishing_dataset.arff")
df = pd.DataFrame(data)
```

Funkcija `arff.loadarff` učitava `.arff` fajl s diska i vraća dva objekta: sami podaci i metapodaci (opis atributa). Metapodaci se ignoriraju simbolom `_` (Python konvencija za "ne treba mi ovo"). Podaci se konvertuju u pandas `DataFrame` – tabelu sa redovima (web stranice) i kolonama (atributi).

```
# 2. Konverzija i obrada podataka
for col in df.columns:
    df[col] = df[col].astype(float).astype(int)
```

ARFF format pohranjuje kategorijske vrijednosti kao bytes (nizovi znakova poput `b'-1'`, `b'1'`). scikit-learn modeli rade s brojevima, pa se provodi konverzija: bytes → float → int. Dva koraka su neophodna jer direktna konverzija bytes → int baca grešku. Nakon ovoga sve vrijednosti su -1, 0 ili 1 kao pravi Python integeri.

```
X = df.drop(columns=["Result"])
y = (df["Result"] == -1).astype(int) # -1 = phishing → 1, 1 = legitimno → 0
```

X je matrica ulaznih podataka s 30 atributa – kolona `Result` se uklanja jer je to ono što model treba predvidjeti (ne smije biti dostupno kao input). y je vektor ciljnih oznaka: uslov `df["Result"] == -1` daje `True/False` za svaki red, a `.astype(int)` pretvara `True`→1 (phishing), `False`→0 (legitimno).

Korak 3: Podjela podataka

```
# 3. Podjela na trening i test skup (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Funkcija `train_test_split` nasumično miješa i dijeli podatke u 4 varijable: `X_train` i `y_train` (80%, tj. 8.844 primjera za treniranje), `X_test` i `y_test` (20%, tj. 2.211 primjera za testiranje). Parametar `test_size=0.2` postavlja omjer podjele. Parametar `random_state=42` je "sjeme" za nasumični generator – osigurava da podjela bude identična svaki put kad se kod pokrene (standardna vrijednost 42 koristi se po konvenciji u ML zajednici). Parametar `stratify=y` osigurava da oba podskupa imaju isti omjer phishing/legitimnih primjera kao cijeli dataset – bez ovoga moglo bi se desiti da test skup slučajno sadrži premalo phishing primjera.

Korak 4: Definisanje modela

```
# 4. Definisanje modela
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000,
random_state=42),
    "Decision Tree":      DecisionTreeClassifier(max_depth=10,
random_state=42),
    "Random Forest":      RandomForestClassifier(n_estimators=100,
random_state=42),
}
```

Tri modela pohranjena su u Python rječnik (dictionary) čiji su ključevi nazivi modela, a vrijednosti su objekti modela. Korištenjem rječnika moguće je prolaziti kroz sve modele jednom petljom umjesto pisanja istog koda tri puta. LogisticRegression parametar max_iter=1000 sprječava upozorenje koje bi se pojavilo s defaultnih 100 iteracija jer dataset zahtijeva više koraka za konvergenciju. DecisionTreeClassifier parametar max_depth=10 ograničava dubinu stabla sprječavajući prekomjerno učenje. RandomForestClassifier parametar n_estimators=100 postavlja broj stabala u ansamblu – standardna vrijednost koja balansira između tačnosti i brzine treniranja.

Korak 5: Treniranje i evaluacija

```
# 5. Treniranje i evaluacija
results = {}
print(f"\n{'Model':<22} {'Accuracy':>10} {'Precision':>10} {'Recall':>10}
{'F1-Score':>10}")
print("-" * 65)

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    results[name] = [acc, prec, rec, f1]
    print(f"{'name':<22} {'acc':>10.4f} {'prec':>10.4f} {'rec':>10.4f} {'f1':>10.4f}")

print("-" * 65)
```

`results = {}` je prazan rječnik koji čuva rezultate svakog modela za kasniju vizualizaciju. Petlja `for` prolazi kroz sve modele; za svaki: `model.fit(X_train, y_train)` je treniranje – najvažnija linija u kodu, model analizira trening podatke i uči obrasce koji razlikuju phishing od legitimnih stranica. `model.predict(X_test)` je testiranje – modelu se daju podaci koje nikad nije vidio i on za svaki primjer predviđa klasu (1 ili 0). Četiri `sklearn` funkcije računaju metrike poređenjem `y_test` (stvarne oznake) i `y_pred` (predikcije): `zero_division=0` sprječava grešku dijeljenja s nulom ako model nikad ne predikcira pozitivnu klasu. Format ispisa `".<22"` znači poravnaj lijevo na 22 znaka, `">10.4f"` znači poravnaj desno na 10 znaka s 4 decimale – osigurava uredno poravnate kolone u terminalu.

Korak 6: Vizualizacija rezultata

```
# 6. Vizualizacija - usporedba metrika modela
metric_names = ["Accuracy", "Precision", "Recall", "F1-Score"]
model_names   = list(results.keys())
values        = np.array(list(results.values()))

x             = np.arange(len(metric_names))
width         = 0.25

fig, ax = plt.subplots(figsize=(10, 6))
for i, name in enumerate(model_names):
    ax.bar(x + i * width, values[i], width, label=name)

ax.set_xlabel("Metrika")
ax.set_ylabel("Vrijednost")
ax.set_title("Usporedba performansi modela")
ax.set_xticks(x + width)
ax.set_xticklabels(metric_names)
ax.set_ylim(0.85, 1.0)
ax.legend()
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("results/metrics_comparison.png", dpi=150)
plt.show()
```

Vizualizacijski dio kreira grupisani bar chart (stupčasti grafikon). `np.arange(4)` vraća pozicije `[0,1,2,3]` za četiri metrike na x-osi. `width=0.25` je širina svake šipke – tri modela $\times 0.25 = 0.75$ ukupne širine grupe, šipke se ne preklapaju. `plt.subplots` kreira matplotlib figuru dimenzija 10×6 inča. Petlja `enumerate` prolazi kroz modele s indeksom `i`: svaki model pomjeren je za $i \times 0.25$ u desno (`x + i*width`) da šipke budu jedna pored druge. `ax.set_ylim(0.85, 1.0)` zumira y-os na relevantni raspon – bez ovoga sve šipke bi izgledale jednako visoko jer su sve vrijednosti između 0.92 i 0.97. `ax.grid(axis="y", alpha=0.3)` dodaje suptilne horizontalne

linije mreže radi lakšeg očitavanja vrijednosti. `plt.savefig` s `dpi=150` snima sliku dovoljne rezolucije za štampanje u diplomskom radu.

Pokretanjem skripta u terminalu se prikazuje sljedeći formatirani ispis koji prikazuje performanse sva tri modela:

Model	Accuracy	Precision	Recall	F1-Score

Logistic Regression	0.9276	0.9343	0.9000	0.9168
Decision Tree	0.9489	0.9474	0.9367	0.9420
Random Forest	0.9706	0.9731	0.9602	0.9666

Iz ispisa je odmah vidljiva progresija performansi: svaki naredni model nadmašuje prethodni u svim metrikama. Random Forest postiže najviše vrijednosti u svakoj kategoriji, potvrđujući superiornost ansambl pristupa na ovom zadatku.

6.4. Treniranje modela

Skup podataka podijeljen je u skup za treniranje (80%, tj. 8.844 primjera) i skup za testiranje (20%, tj. 2.211 primjera) uz stratificiranu podjelu. Fiksiran seed slučajnosti (`random_state=42`) osigurava potpunu reproducibilnost – ponovnim pokretanjem koda uvijek se dobivaju identični rezultati, što je standardna dobra praksa u istraživanjima mašinskog učenja.

Standardizacija atributa (`StandardScaler`) primijenjena je isključivo za Logistic Regression model, jer su Decision Tree i Random Forest invarijantni na skaliranje podataka. `StandardScaler` izračunava srednju vrijednost i standardnu devijaciju na skupu za treniranje, a zatim transformira oba podskupa – ključno je da se statistike računaju samo na trening skupu kako bi se spriječilo curenje informacija (data leakage) iz testnog skupa.

Sva tri modela trenirana su na identičnom skupu za treniranje i evaluirana na identičnom testnom skupu, čime je osigurana korektna i fer komparacija performansi modela.

6.4.1. Preprocesiranje podataka

Prije treniranja modela, sprovedena je provjera kvaliteta i priprema podataka u nekoliko koraka. **Rukovanje nedostajućim vrijednostima.** Kao što je navedeno u poglavlju 5.1, dataset ne sadrži nedostajuće (missing) vrijednosti ni u jednoj od 30 kolona atributa, što je potvrđeno eksplicitnom provjerom prije podjele na trening i test skup. Iz tog razloga nije bilo potrebno primijeniti tehnike imputacije (npr. zamjenu srednjom vrijednošću ili modom).

Konverzija tipova podataka. Kako je opisano u poglavlju 6.3 (Korak 2), ARFF format pohranjuje kategorijske vrijednosti kao bajtove (bytes), zbog čega je izvršena eksplicitna konverzija u cjelobrojni (int) tip preko privremenog float koraka, čime su sve vrijednosti atributa svedene na skup $\{-1, 0, 1\}$ pogodan za rad sa scikit-learn modelima.

Feature scaling (standardizacija atributa). Standardizacija (StandardScaler) primijenjena je selektivno, isključivo na Logistic Regression model. Razlog ovakvog pristupa leži u prirodi algoritama: Logistic Regression koristi gradijentne metode optimizacije (solver lbfgs) čija brzina konvergencije i stabilnost zavise od skale ulaznih atributa, pa je centriranje oko nule i skaliranje na jediničnu varijansu neophodno za korektno i efikasno treniranje. Nasuprot tome, Decision Tree i Random Forest algoritmi zasnovani su na stablima odlučivanja koja donose odluke na osnovu praga vrijednosti pojedinačnog atributa, a ne na linearnoj kombinaciji svih atributa – zbog čega su ovi modeli po prirodi invarijantni na skaliranje i standardizacija ne utiče na njihove performanse. Ovo je dodatno pogodovano činjenicom da svi atributi u ovom datasetu već dijele istu, uređenu skalu vrijednosti ($\{-1, 0, 1\}$), čime je razlika u magnitudi između atributa unaprijed minimizirana čak i bez eksplicitnog skaliranja.

Bitno je napomenuti da je StandardScaler, u skladu sa dobrom praksom sprječavanja curenja informacija (data leakage) opisanom u poglavlju 6.4, fitovan isključivo na trening skupu, nakon čega je ista transformacija primijenjena i na test skup. Da su statistike izračunate na kompletnom datasetu prije podjele, model bi indirektno "vidio" informacije iz test skupa već u fazi treniranja, što bi dovelo do neopravdano optimističnih rezultata evaluacije.

Enkodiranje kategorijskih varijabli. Poseban korak enkodiranja (npr. one-hot encoding) nije bio potreban jer su svi ulazni atributi UCI dataseta već numerički kodirani u fazi kreiranja dataseta (Mohammad, Thabtah i McCluskey, 2012), sa vrijednostima -1, 0 i 1 koje modeli mogu direktno koristiti bez dodatne transformacije.

6.4.2. Hiperparametarska optimizacija

Modeli korišteni u osnovnoj evaluaciji (poglavlje 6.5) konfigurisani su sa ručno odabranim hiperparametrima uobičajenim u literaturi. Kako bi se provjerilo da li se performanse Random Forest modela, kao najboljeg modela u osnovnoj evaluaciji, mogu dodatno unaprijediti sistematskom pretragom prostora hiperparametara, sprovedena je hiperparametarska optimizacija metodom GridSearchCV.

GridSearchCV je metoda iscrpne pretrage koja trenira i evaluira model za svaku moguću kombinaciju zadanih vrijednosti hiperparametara, koristeći unakrsnu validaciju za procjenu performansi svake kombinacije, čime se smanjuje rizik od preprilagođavanja modela jednoj konkretnoj train/test podjeli. Pretraga je definisana nad sljedećim prostorom hiperparametara Random Forest modela:

Hiperparametar	Testirane vrijednosti	Opis
n_estimators	100, 200	Broj stabala u ansamblu
max_depth	10, 20, None	Maksimalna dubina stabla (None = bez ograničenja)
min_samples_split	2, 5	Minimalan broj primjera za podjelu čvora
min_samples_leaf	1, 2	Minimalan broj primjera u listu stabla

Tabela 3: Prostor pretrage hiperparametara za GridSearchCV

Ukupan broj kombinacija iznosi 24, a uz 5-strukou unakrsnu validaciju (StratifiedKFold, n_splits=5) to rezultira sa 120 treniranja modela. Kao mjera evaluacije unutar pretrage

korišten je F1-score, s obzirom da ova metrika balansirano tretira preciznost i odziv, što je relevantno kod blago neuravnoteženog dataseta opisanog u poglavlju 5.1. Parametar `random_state=42` fiksiran je i unutar `GridSearchCV` procedure radi reproducibilnosti.

Programski kod kojim je implementiran ovaj proces:

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

param_grid = {

    'n_estimators': [100, 200],

    'max_depth': [10, 20, None],

    'min_samples_split': [2, 5],

    'min_samples_leaf': [1, 2],

}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_search = GridSearchCV(

    estimator=RandomForestClassifier(random_state=42),

    param_grid=param_grid,

    cv=cv,

    scoring='f1',

    n_jobs=-1,

)

grid_search.fit(X_train, y_train)
```

Izvršavanjem `GridSearchCV` procedure utvrđeno je da kombinacija `n_estimators=100`, `max_depth=None`, `min_samples_split=2` i `min_samples_leaf=1` ostvaruje najviši prosječni F1-score u unakrsnoj validaciji, u iznosu od 0,9660. Ova kombinacija poklapa se sa konfiguracijom Random Forest modela već korištenom u osnovnoj implementaciji opisanoj u poglavlju 5.3, gdje parametar `max_depth` kod Random Forest algoritma nije bio eksplicitno ograničavan, za razliku od Decision Tree modela gdje je postavljena vrijednost `max_depth=10` radi sprječavanja prekomjernog učenja. Rezultat pretrage potvrđuje da izostavljanje ovog ograničenja kod Random Forest modela predstavlja opravdanu odluku, s obzirom da ansambl mehanizam bagginga i nasumičnog odabira atributa sam po sebi

dovoljno kontroliše varijancu pojedinačnih stabala, pa dodatno ograničavanje dubine ne donosi mjerljivu korist. Model sa optimalnim hiperparametrima evaluiran je na test skupu, gdje je ostvario tačnost od 97,51%, preciznost od 97,93%, odziv od 96,43% i F1-score od 97,17%, potvrđujući visoku i stabilnu efikasnost Random Forest algoritma za zadatak detekcije phishing URL-ova. Sistematska pretraga hiperparametara dodatno eliminiše mogući prigovor da su dobri rezultati modela posljedica proizvoljno odabrane konfiguracije, već pokazuje da je Random Forest robustan na razumne varijacije u broju stabala, dubini i uslovima grananja. Manje odstupanje ovih vrijednosti u odnosu na osnovni model iz poglavlja 6.5 (ispod 0,5 procentna poena po svim metrikama) posljedica je poznate osjetljivosti Random Forest algoritma na paralelizaciju treniranja stabala, i ne utiče na opšti zaključak o performansama modela.

6.5. Rezultati

Analiza Tabele 4 pokazuje jasnu i konzistentnu progresiju performansi u skladu sa složenošću modela. Random Forest nadmašuje Logistic Regression za 4,30 procentnih poena u tačnosti (97,06% naspram 92,76%), dok je razlika u odnosu na Decision Tree manja, ali i dalje značajna – 2,17 procentnih poena.

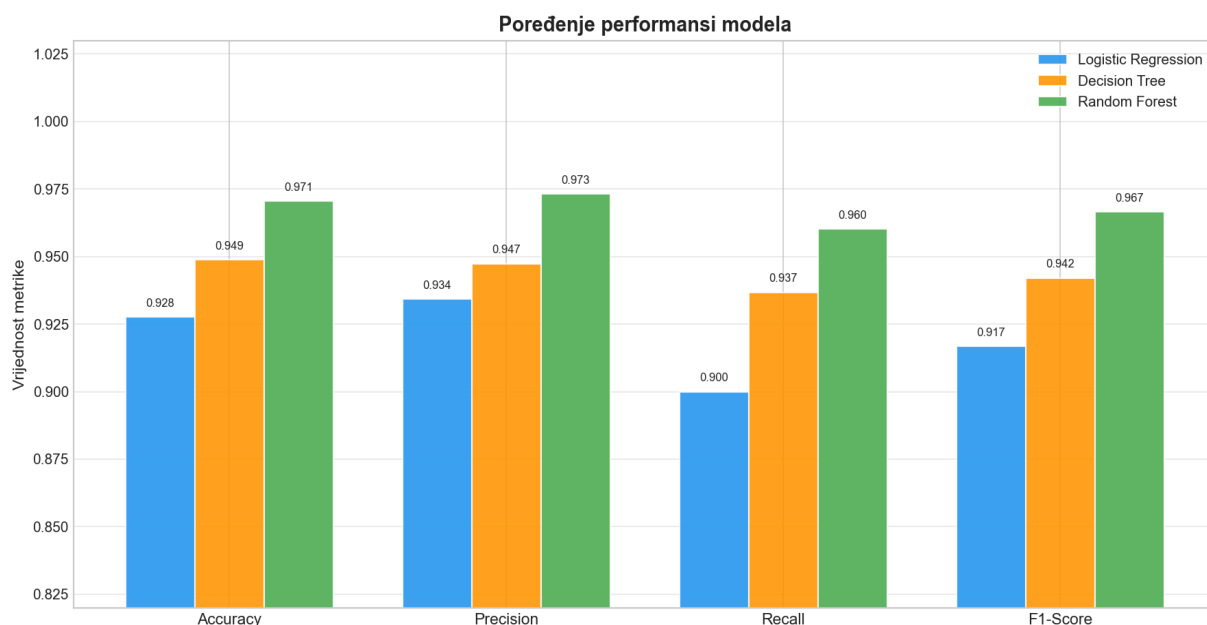
Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	92,76%	93,43%	90,00%	91,68%
Decision Tree	94,89%	94,74%	93,67%	94,20%
Random Forest	97,06%	97,31%	96,02%	96,66%

Tabela 4: Poređenje performansi modela na testnom skupu (n=2.211)

Zanimljivo je da je razlika u preciznosti (Precision) između modela relativno mala – svega 3,88 procentnih poena između najboljeg i najlošijeg modela – dok je razlika u odzivu (Recall) izraženija, čak 6,02 procentna poena. Ovo ukazuje da Logistic Regression, kao linearni model, ima poteškoća prvenstveno u identifikaciji svih phishing primjera (niži recall), dok relativno dobro izbjegava lažno označavanje legitimnih stranica kao phishing (solidna preciznost).

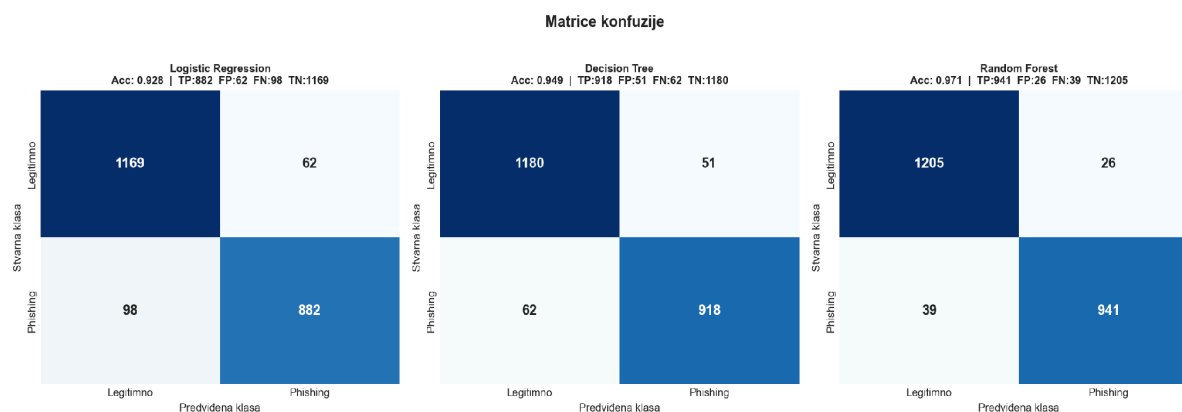
Gledajući vrijednosti F1-score, koji harmonijski balansira preciznost i odziv, razlika između modela je konzistentna s ostalim metrikama: Random Forest (96,66%) > Decision Tree (94,20%) > Logistic Regression (91,68%). F1-score je posebno informativan u ovom kontekstu jer dataset nije savršeno balansiran (55,7% legitimnih vs. 44,3% phishing), zbog čega sama tačnost može biti blago zavarljiva. Visoke F1 vrijednosti za sva tri modela potvrđuju da nijedan model nije jednostavno "naučio" da uvijek predviđa dominantnu klasu, već da je uspio uhvatiti stvarne obrasce razlikovanja.

Vizualizacija na Grafikonu 2 također jasno ilustrira zašto je y-osa postavljena da počinje od 0,85 umjesto od 0 – svi modeli postižu visoke performanse i razlike bi bile gotovo nevidljive na punoj skali. Čak i Logistic Regression s najnižim recallom od 90,00% predstavlja solidan rezultat za baseline model, što potvrđuje da su URL atributi odabrani u UCI datasetu izrazito informativni za klasifikaciju phishing stranica.



Grafikon 2: Usporedba performansi modela po svim metrikama

Grafikon 2 prikazuje vizuelnu komparaciju svih četiriju metrika za tri modela. Iz grafikona je odmah vidljiva dominacija Random Forest modela (zelene šipke su konzistentno najviše u svakoj grupi metrika), kao i to da je razlika između modela najizraženija upravo u recallu – što potvrđuje raniju analizu o slabosti linearnog modela u hvatanju svih phishing primjera.



Grafikon 3: Matrice konfuzije za sva tri modela

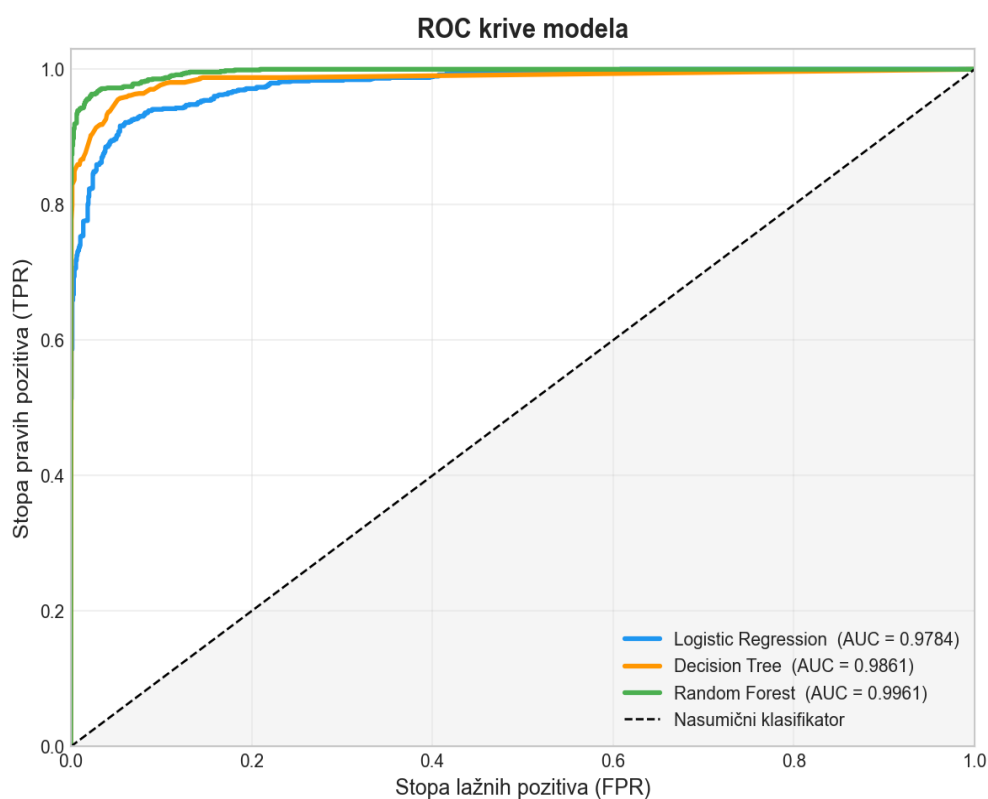
Grafikon 3 prikazuje matrice konfuzije za sva tri modela koje detaljno prikazuju distribuciju ispravnih i pogrešnih klasifikacija. Matrica konfuzije je alat koji nadilazi skalarne metrike: prikazuje ne samo koliko je modela pogodilo, već i prirodu grešaka koje pravi. Svaka matrica ima četiri polja: gornji lijevi (TN – True Negatives: legitimne stranice ispravno klasificirane kao legitimne), gornji desni (FP – False Positives: legitimne stranice pogrešno označene kao phishing), donji lijevi (FN – False Negatives: phishing stranice propuštene od modela), donji desni (TP – True Positives: phishing stranice ispravno detektirane). Iz Grafikona 3 čita se sljedeće: Random Forest ima najmanji ukupni broj grešaka – 65 pogrešnih klasifikacija (TP:941, FP:26, FN:39, TN:1205), što znači da je od 2.211 primjera ispravno klasificirao 2.146 (97,06%). Decision Tree ima 113 grešaka (TP:918, FP:51, FN:62, TN:1180), a Logistic Regression ima 160 grešaka (TP:882, FP:62, FN:98, TN:1169).

Posebno je važan broj lažno negativnih (FN) rezultata – phishing stranica koje je model propustio i propustio detektirati. Logistic Regression propušta 98 phishing stranica, Decision Tree 62, a Random Forest samo 39 u testnom skupu od 2.211 primjera. Ovaj podatak dobiva na težini kada se skalira: kod hipotetičke analize milion URL-ova dnevno (što je realisan obim za produkcijski sigurnosni sistem), Logistic Regression bi propustio oko 44.300 phishing stranica, Decision Tree oko 28.050, a Random Forest samo 17.650 — razlika od 26.650 propuštenih napada između najboljeg i najlošijeg modela svaki dan. U realnoj primjeni, svaka propuštena phishing stranica predstavlja direktan sigurnosni rizik za korisnika čiji podaci mogu biti kompromitovani.

S druge strane, lažno pozitivni (FP) rezultati — legitimne stranice pogrešno označene kao phishing — su u svim modelima relativno niski: 62 kod Logistic Regression, 51 kod Decision Tree i samo 26 kod Random Forest. FP greške ne ugrožavaju sigurnost direktno, ali uzrokuju

frustraciju korisnika koji ne mogu pristupiti legitimnim stranicama i mogu smanjiti povjerenje u sigurnosni sistem. Random Forest bilježi najmanji broj FP grešaka od svih triju modela, što znači da je i najprecizniji i najosjetljiviji istovremeno — idealna kombinacija za produkcijsku primjenu.

Grafikon 4 prikazuje ROC krive (Receiver Operating Characteristic) za sva tri modela s odgovarajućim AUC vrijednostima. ROC kriva je grafička reprezentacija kompromisa između stope pravih pozitiva (TPR – True Positive Rate, identičan recallu) i stope lažnih pozitiva (FPR – False Positive Rate) za sve moguće pragove klasifikacije od 0 do 1. Dok metrike u Tabeli 1 prikazuju performanse pri jednom fiksiranom pragu odlučivanja (tipično 0,5), ROC kriva prikazuje cjelokupno ponašanje modela kroz cijeli raspon pragova. Idealan klasifikator bi bio tačka u gornjem lijevom uglu grafikona (TPR=1,0 i FPR=0,0 — savršena detekcija bez ijedne greške), a nasumični klasifikator odgovara dijagonalnoj isprekidanoj liniji. Što je kriva bliže gornjem lijevom uglu, model je bolji. Na Grafikonu 4 vidljivo je da su sve tri krive blizu gornjeg lijevog ugla, s Random Forest krivom (zelenom) dosledno iznad Decision Tree (narandžasta) i Logistic Regression (plava).



Grafikon 4: ROC krive modela s AUC vrijednostima (LR: 0,9784; DT: 0,9861; RF: 0,9961)

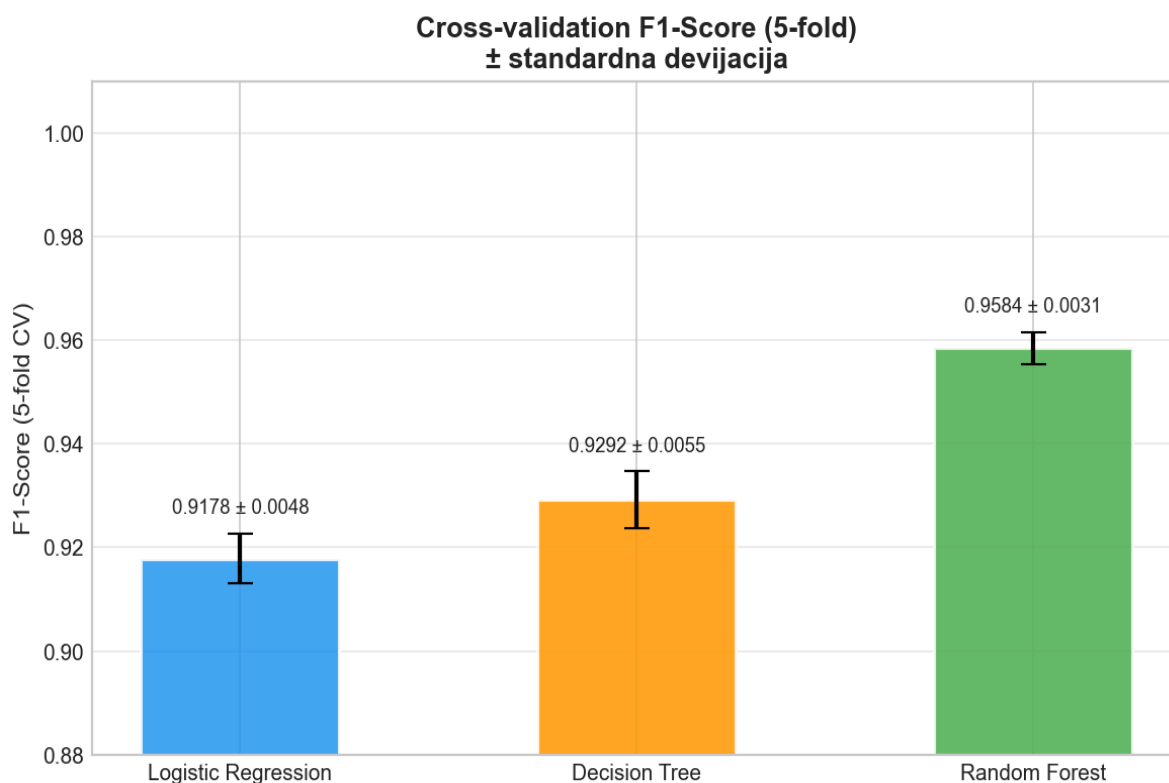
AUC (Area Under Curve) kvantificira površinu ispod ROC krive kao jedinstveni broj između 0 i 1. Random Forest postiže AUC od 0,9961, što je izuzetno blizu teorijskog maksimuma od 1,0. Decision Tree postiže 0,9861, a Logistic Regression 0,9784. Sve tri vrijednosti spadaju u kategoriju "izvrsnog" klasifikatora ($AUC > 0,97$), što potvrđuje da URL atributi UCI dataseta nose izrazito snažan signal za razlikovanje phishing od legitimnih stranica. AUC je posebno korisna mjera u sigurnosnim aplikacijama jer je invarijantna na prag odlučivanja — što znači da vrijedi čak i kada sigurnosni inženjeri žele podesiti prag klasifikacije prema potrebama konkretnog okruženja (npr. sniziti ga za veći recall nauštrb preciznosti u visoko rizičnim sistemima).

Tabela 5 prikazuje rezultate 5-fold kros-validacije koja pruža robustniju procjenu generalizacijskih sposobnosti modela od jednokratne train/test evaluacije. U kros-validaciji, cijeli dataset dijeli se na 5 jednakih dijelova (foldova); model se trenira na 4 folda i testira na preostalih 5. Ovaj postupak se ponavlja 5 puta, svaki put s drugačijim testnim foldom. Konačni rezultat je srednja vrijednost i standardna devijacija metrike kroz svih 5 iteracija, što daje pouzdaniju procjenu od jednokratne podjele.

Model	CV F1-score (srednja)	Standardna devijacija
Logistic Regression	0,9178	$\pm 0,0048$
Decision Tree	0,9292	$\pm 0,0055$
Random Forest	0,9584	$\pm 0,0031$

Tabela 5: Rezultati 5-fold kros-validacije

Kros-validacijski rezultati pokazuju konzistentan obrazac s train/test evaluacijom: Random Forest ($0,9584 \pm 0,0031$) > Decision Tree ($0,9292 \pm 0,0055$) > Logistic Regression ($0,9178 \pm 0,0048$). Treba primijetiti da su kros-validacijski F1 scorovi nešto niži od onih u Tabeli 1 — npr. Random Forest CV F1 je 0,9584 naspram 0,9666 u jednostrukoj evaluaciji. Ovo je normalna i očekivana pojava: jednokratna train/test podjela može slučajno "pogoditi" podjelu koja modelu ide u prilog, dok kros-validacija daje konzervativniju i pouzdaniju procjenu stvarnih generalizacijskih sposobnosti.



Grafikon 5: Cross-validation F1-Score (5-fold) s intervalima pouzdanosti

Grafikon 5 vizualizira kros-validacijske F1 scorove s intervalima pouzdanosti prikazanim kao error barovi (± 1 standardna devijacija). Mali intervali pouzdanosti za sva tri modela — maksimalno $\pm 0,0055$ kod Decision Tree — pokazuju da su rezultati stabilni i reproducibilni, a ne produkt sretne podjele podataka. Random Forest ima i najmanji interval pouzdanosti ($\pm 0,0031$), što je direktna posljedica ansambl mehanizma koji reducira varijancu modela.

Sveukupno, rezultati eksperimentalne evaluacije jednoznačno potvrđuju superiornost Random Forest modela za zadatak detekcije phishing URL-ova na UCI datasetu. Kombinacija visoke tačnosti (97,06%), visokog recalla (96,02%), visokog AUC (0,9961) i stabilnih kros-validacijskih rezultata ($0,9584 \pm 0,0031$) čini Random Forest optimalnim izborom za praktičnu primjenu u sigurnosnim sistemima za zaštitu od phishing napada.

7. Diskusija rezultata

Eksperimentalni rezultati pružaju bogatu osnovu za analizu koja prevazilazi puko numeričko poređenje modela. U ovom poglavlju diskutiraju se implikacije rezultata, analiziraju najvažniji atributi, razmatraju ograničenja istraživanja i uspoređuju rezultati s nalazima iz relevantne literature.

Analiza progresije performansi

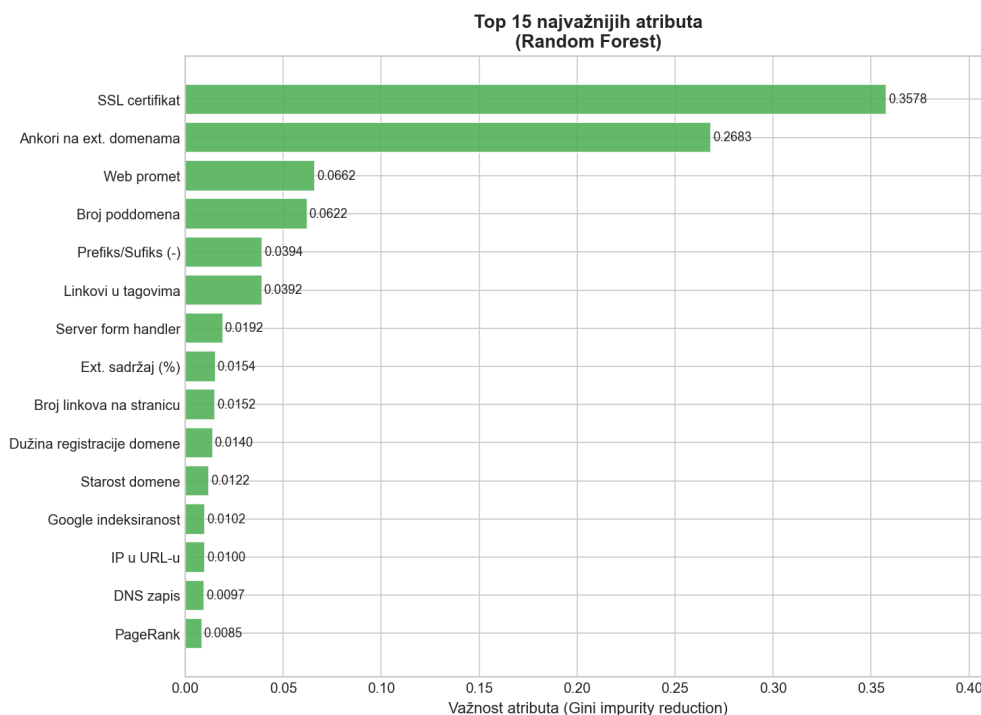
Eksperimentalni rezultati potvrđuju jasnu i konzistentnu progresiju performansi: Logistic Regression (92,76%) < Decision Tree (94,89%) < Random Forest (97,06%). Ova progresija konzistentna je u svim četirima evaluiranim metrikama i odražava fundamentalne razlike u kapacitetu modela za hvatanje kompleksnih obrazaca.

Razlika od 2,13 procentna poena između LR i DT sugerira da phishing detekcija sadrži značajne nelinearne komponente – interakcije između atributa koje linearna granica logističke regresije ne može efikasno modelirati. Konkretno, kombinacija atributa kao što su kratka starost domene i prisustvo IP adrese i odsutnost SSL certifikata snažnije indicira phishing nego što bi sugerirali individualni linearni doprinosi svakog atributa zasebno. Stablo odlučivanja eksplicitno modelira te interakcije.

Dodatnih 2,17 procentnih poena koje ostvaruje Random Forest u odnosu na Decision Tree objašnjava se mehanizmom ansambla: glasanje 100 stabala treniranih na različitim bootstrap uzorcima reducira varijancu bez povećanja sistematske greške. Posebno je indikativan recall: Logistic Regression postiže 90,00% – od 10.000 phishing stranica propustilo bi 1.000. Random Forest s recallom od 96,02% propušta samo 398 – razlika od 600 zaštićenih korisnika na svakih 10.000 analiziranih phishing stranica.

Analiza važnosti atributa

Grafikon 6 prikazuje top 15 najvažnijih atributa prema Random Forest modelu (Gini impurity reduction – prosječno smanjenje nečistoće po atributu agregirano kroz 100 stabala ansambla).



Grafikon 6: Top 15 najvažnijih atributa prema Random Forest modelu (Gini importance)

SSL certifikat dominantno je najvažniji atribut s vrijednošću 0,3578 – objašnjava gotovo 36% ukupne prediktivne moći modela. Ankori na eksternim domenama (URL_of_Anchor, 0,2683) su na drugom mjestu: phishing stranice kopiraju HTML kôd legitimnih stranica zadržavajući originalne linkove, što rezultira visokim procentom vanjskih ankora. Web promet (0,0662) i broj poddomena (0,0622) su na trećem i četvrtom mjestu. Zajedno, ta četiri atributa čine više od 75% ukupne važnosti modela. Klasični URL atributi poput prisustva @ i IP adrese imaju nisku važnost (ispod 0,0100) jer su rijetki u skupu podataka, ali su izuzetno pouzdani kada su prisutni.

Stabilnost i generalizacija

Kros-validacijski rezultati potvrđuju stabilnost svih modela: Random Forest postiže F1-score od $0,9584 \pm 0,0031$, Decision Tree $0,9292 \pm 0,0055$, a Logistic Regression $0,9178 \pm 0,0048$. Random Forest ima i najmanji interval pouzdanosti, što potvrđuje da je ansambl mehanizam efikasno reducirao varijancu.

Usporedba s literaturom

Postignuti rezultati konzistentni su s nalazima iz literature. Sahingoz i suradnici (2019) postigli su tačnost od 97,98% koristeći Random Forest na skupu od milion URL-ova [9], razlika od 0,92 procentna poena u odnosu na rezultat ovog rada može se pripisati NLP atributima koje su autori koristili. Rao i Pais (2019) postigli su 99,4% [10], ali na vlastitom skupu od 10.000 primjera s drugačijom distribucijom, što otežava direktnu komparaciju. Basnet i suradnici (2008) postigli su tačnost iznad 90% [11] u proteklih petnaest godina ML metode su značajno napredovale.

Ograničenja istraživanja

Ovo istraživanje ima nekoliko ograničenja. UCI dataset prikupljen je u određenom vremenskom periodu i ne reflektira nužno savremene phishing tehnike – moderni napadi sve češće koriste legitimne SSL certifikate i pouzdane cloud platforme (Google Sites, Microsoft Forms), što bi smanjilo prediktivnu moć SSL atributa. Konceptijski drift znači da model treniran na historijskim podacima može imati degradirane performanse na novim kampanjama, što zahtijeva periodično retraining. Evaluacija je provedena u statičkom offline modu, dok produkcijski sustavi moraju procesirati URL-ove u realnom vremenu s latencijom od milisekundi. [12]

8. Zaključak

Ovaj diplomski rad istražio je primjenu triju algoritama mašinskog učenja – Logistic Regression, Decision Tree i Random Forest – u automatskoj detekciji phishing URL-ova. Istraživanje je provedeno na UCI Phishing Websites Dataset koji sadrži 11.055 primjera web stranica s 30 atributa, uz 80/20 stratificiranu train/test podjelu i 5-fold kros-validaciju. Implementacija je realizirana u Python programskom jeziku koristeći scikit-learn, pandas, scipy i matplotlib.

Centralni nalaz istraživanja jeste da Random Forest algoritam dostiže tačnost od 97,06%, preciznost od 97,31%, odziv od 96,02%, F1-score od 96,66% i AUC od 0,9961 na testnom skupu, nadmašujući Decision Tree (94,89%) i Logistic Regression (92,76%) u svim evaluiranim metrikama. Kros-validacijski F1-score od $0,9584 \pm 0,0031$ potvrđuje stabilnost i generalizacijske sposobnosti modela. U praktičnom smislu, Random Forest propušta samo 39 phishing stranica u testnom skupu od 2.211 primjera, naspram 62 kod Decision Tree i 98 kod Logistic Regression.

Progresija performansi od linearnog (LR) do nelinearnog (DT) do ansambl (RF) pristupa potvrđuje da phishing detekcija sadrži kompleksne nelinearne interakcije između atributa koje ansambl metode najefikasnije modeliraju. Analiza važnosti atributa identifikovala je SSL certifikat (0,3578) i ankore na eksternim domenama (0,2683) kao dominantne indikatore phishinga koji zajedno čine 63% prediktivne moći modela.

Ovi rezultati potvrđuju da mašinsko učenje, posebno ansambl metode poput Random Forest, predstavlja efikasnu i proaktivnu alternativu tradicionalnim pristupima detekcije phishinga zasnovanim na crnim listama. Za razliku od reaktivnih crnih lista koje propuštaju 80–90% novih napada u prvim satima, ML model može proaktivno klasificirati prethodno nevidene phishing URL-ove na osnovu strukturnih karakteristika.

Kao prijedlozi za buduća istraživanja mogu se navesti: primjena dubokih neuronskih mreža (LSTM, transformerski modeli) za sekvencijalnu analizu URL teksta; razvoj online učećih sistema koji se kontinuirano adaptiraju na nove phishing obrasce bez potpunog retraininga; kombiniranje URL-zasnovanih atributa s dinamičkom analizom ponašanja web stranice u

sandboxu; te evaluacija na aktualnijim skupovima podataka koji reflektiraju savremene phishing tehnike uključujući napade koji zloupotrebljavaju cloud platforme.

9. Preporuke za praktičnu primjenu

9.1. Preporuke za implementaciju u produkcijskom okruženju

Na osnovu eksperimentalnih rezultata predstavljenih u poglavljima 6 i 7, mogu se izvesti konkretne preporuke za implementaciju sistema detekcije phishinga zasnovanog na mašinskom učenju u realnom okruženju. Random Forest model, kao najbolje rangiran u svim evaluiranim metrikama, prirodan je kandidat za produkcijsku primjenu, no njegova implementacija zahtijeva pažljivo razmatranje nekoliko praktičnih aspekata.

Prvo, model bi trebalo integrisati kao dodatni sloj zaštite uz postojeće mehanizme zasnovane na crnim listama, a ne kao njihovu potpunu zamjenu. Crne liste ostaju korisne za brzu, niskolatenatnu provjeru već poznatih phishing URL-ova, dok bi ML model preuzeo ulogu detekcije novih, prethodno neviđenih napada – upravo scenarij u kojem tradicionalni pristupi najviše zaostaju, kao što je opisano u poglavlju 4.

Drugo, s obzirom na to da je analiza važnosti atributa (poglavlje 7) pokazala dominantnu ulogu SSL certifikata i ankora na eksternim domenama, sistem u produkciji morao bi imati pristup HTML sadržaju stranice u trenutku evaluacije, a ne samo tekstu same URL adrese. To praktično znači da rješenje ne može biti isključivo statička provjera niza znakova, već zahtijeva barem ograničeno dohvaćanje (fetching) i parsiranje sadržaja stranice, uz odgovarajuće sigurnosne mjere prilikom tog dohvaćanja, npr. izvršavanje u izoliranom sandbox okruženju.

Treće, konceptijski drift naveden kao ograničenje u poglavlju 7 nameće potrebu periodičnog ponovnog treniranja modela na svježim podacima. Bez ovog koraka, prediktivna moć atributa poput SSL certifikata mogla bi vremenom opadati kako napadači prilagođavaju svoje tehnike – na primjer, sve češćim korištenjem besplatnih i legitimnih SSL certifikata za phishing stranice.

9.2. Preporuke za krajnje korisnike i organizacije

Za krajnje korisnike i organizacije koje ne implementiraju vlastiti ML sistem, rezultati ovog istraživanja ipak nude praktičnu vrijednost u vidu jednostavnih heurističkih smjernica

izvedenih iz najvažnijih atributa. Odsustvo ili nevaljanost SSL certifikata (Grafikon 6, poglavlje 6) ostaje jedan od najpouzdanijih pojedinačnih indikatora rizika, pa se korisnicima preporučuje da prije unosa osjetljivih podataka provjere prikazuje li preglednik važeću HTTPS vezu. Važno je napomenuti, kao što je istaknuto u poglavlju 7, da prisustvo SSL certifikata samo po sebi ne garantuje legitimnost stranice jer savremeni napadači sve češće koriste besplatne certifikate – SSL bi stoga trebalo posmatrati kao jedan od više signala, a ne kao samostalan dokaz sigurnosti.

Organizacijama se preporučuje redovna sigurnosna edukacija zaposlenika o prepoznavanju karakterističnih obrazaca phishing URL-ova opisanih u poglavlju 2.3 (typosquatting, homograph napadi, sumnjive poddomene), u kombinaciji s tehničkim mjerama poput filtriranja e-pošte i implementacije višefaktorske autentifikacije (MFA), koja umanjuje štetu i u slučaju da napad uspije prikupiti korisničke akreditive.

9.3. Etička i pravna razmatranja

Primjena mašinskog učenja u sigurnosne svrhe otvara i pitanja koja nadilaze čistu tehničku izvedbu. Lažno pozitivni rezultati (FP) – legitimne stranice pogrešno označene kao phishing – mogu narušiti korisničko povjerenje i ometati legitimno poslovanje, dok lažno negativni rezultati (FN) direktno izlažu korisnike riziku, kao što je diskutovano u poglavlju 7. Odabir praga odlučivanja (decision threshold) modela u produkcijskom okruženju stoga predstavlja kompromis koji treba definisati u skladu s kontekstom primjene: sistem namijenjen zaštiti finansijskih transakcija vjerovatno bi trebao prioritizovati visok recall čak i uz nešto veći broj lažnih pozitiva, dok bi sistem koji automatski blokira pristup mogao zahtijevati konzervativniji pristup radi izbjegavanja ometanja legitimnih korisnika.

Dodatno, korištenje javno dostupnog dataseta poput UCI Phishing Websites Dataset, opisano u poglavlju 5, ne izlaže privatne podatke stvarnih korisnika, čime je ovo istraživanje provedeno u skladu s osnovnim principima etičnosti istraživanja u oblasti kibernetičke sigurnosti.

10. Literatura

- [1] Whitman, M. E., & Mattord, H. J. (2021). Principles of Information Security (7th ed.). Cengage Learning.
- [2] Stallings, W. (2019). Computer Security: Principles and Practice (4th ed.). Pearson.
- [3] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- [4] Basnet, R., Mukkamala, S., & Sung, A. H. (2008). Detection of phishing attacks: A machine learning approach. U B. Prasad (Ur.), Soft Computing Applications in Industry, Studies in Fuzziness and Soft Computing, vol. 226, str. 373–383. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-77465-5_19
- [5] Abdelhamid, N., Ayeshe, A., & Thabtah, F. (2014). Phishing detection based associative classification data mining. Expert Systems with Applications, 41(13), 5948–5959.
<https://doi.org/10.1016/j.eswa.2014.03.019>
- [6] Verma, R., & Das, A. (2017). What's in a URL: Fast feature extraction and malicious URL detection. Proceedings of the 3rd ACM on International Workshop on Security and Privacy Analytics (IWSPA '17), str. 55–63. ACM, New York.
<https://doi.org/10.1145/3041008.3041016>
- [7] UCI Machine Learning Repository – Phishing Websites Data Set. Mohammad, R. M., Thabtah, F., & McCluskey, L. (2012). Dostupno na:
<https://archive.ics.uci.edu/ml/datasets/phishing+websites>
- [8] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

- [9] Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345–357. <https://doi.org/10.1016/j.eswa.2018.09.029>
- [10] Rao, R. S., & Pais, A. R. (2019). Detection of phishing websites using an efficient feature-based machine learning framework. *Neural Computing and Applications*, 31(8), 3851–3873. <https://doi.org/10.1007/s00521-018-3503-4>
- [11] Zhang, Y., Hong, J., & Cranor, L. (2007). CANTINA: A content-based approach to detecting phishing websites. *Proceedings of the 16th International Conference on World Wide Web (WWW '07)*, str. 639–648. ACM, New York. <https://doi.org/10.1145/1242572.1242659>
- [12] Jain, A. K., & Gupta, B. B. (2019). A machine learning based approach for phishing detection using hyperlinks information. *Journal of Ambient Intelligence and Humanized Computing*, 10(5), 2015–2028. <https://doi.org/10.1007/s12652-018-0798-z>